

School of Technology and Research

S.Y. B.TECH (SEMESTER-III) • ACADEMIC YEAR 2026-27 • COMPLETE LENGTHY ANSWER BANK (UNITS I & II)

Unit-I: Introduction to Data Structures and Algorithm Analysis

Q1. Define data, information, and knowledge with suitable examples. Also, explain how data structures play a vital role in organizing and processing data efficiently.

1. Definitions and Examples:

- **Data:** Data represents raw, unorganized, and unprocessed facts, symbols, observations, or measurements without context or semantic meaning. For example, the numbers **25**, **40**, and **12** are raw data points.
- **Information:** Information is processed, structured, organized, and contextualized data that provides meaning and makes it useful for decision-making. For example, stating that "The temperatures recorded in Pune on consecutive days are 25°C, 40°C, and 12°C" transforms the raw numbers into meaningful information.
- **Knowledge:** Knowledge is the human understanding, interpretation, and synthesis of information gained through experience, reasoning, and study, enabling actionable insights. For instance, knowing that "40°C indicates an extreme heatwave requiring specific precautionary measures and grid load management" is knowledge.

2. Role of Data Structures in Organizing and Processing Data:

A data structure is a specialized format for organizing, managing, storing, and retrieving data in a computer's memory or auxiliary storage. They are crucial because:

- **Efficiency & Speed:** They optimize time and space complexities for operations like searching, insertion, deletion, and sorting.
- **Data Organization:** Real-world entities have complex relationships (hierarchical, sequential, network). Data structures map these relationships accurately into computer memory.
- **Resource Management:** Efficient data structures minimize memory overhead, allowing large datasets to be processed within hardware constraints.

Q2. Apply both pseudo-code and flowchart representations to design an algorithm for finding the largest of three numbers. Compare the two representations.

1. Pseudo-code Representation:

```
Algorithm FindLargest(a, b, c)
Input: Three numbers a, b, and c
Output: The largest among a, b, and c
Begin
    If a >= b and a >= c then
        Return a
    Else if b >= a and b >= c then
        Return b
    Else
        Return c
    End If
End
```

2. Flowchart Representation (Conceptual Structure):

A flowchart uses geometric symbols connected by arrows. It starts with an oval "Start", a parallelogram for input (*a, b, c*), diamond decision boxes evaluating *a ≥ b* and *a ≥ c*, process boxes, output parallelograms, and ends with an oval "Stop".

3. Comparison:

Feature	Pseudo-code	Flowchart
Nature	Text-based, resembles programming syntax.	Graphical, pictorial representation.
Ease of Modification	Easy to edit textually; highly flexible.	Tedious to redraw and adjust connections.
Best Suited For	Complex algorithms and direct translation to code.	Visualizing logic flow and explaining concepts to beginners.

Q3. What is an Abstract Data Type (ADT)? Explain the concept with a suitable example.

1. Concept of Abstract Data Type (ADT):

An Abstract Data Type is a mathematical model or logical description of a data type defined by its behavior (semantics) from the user's point of view. It specifies *what* operations can be performed and *what* data values are stored, but hides *how* these operations are implemented or how the data is stored in memory (encapsulation and information hiding).

2. Example: The Stack ADT

A Stack is a Last-In-First-Out (LIFO) data structure. Its ADT specification includes:

- **Data Elements:** A collection of ordered elements.
- **Operations:** `push(item)` (adds item to top), `pop()` (removes and returns top item), `peek()` (returns top item without removing), `isEmpty()` (checks if stack has no elements).

Whether implemented using a contiguous array or a linked list, the user interacts with the stack using the exact same interface, completely agnostic of the underlying memory representation.

Q4. Compare and justify, with real-world examples, the distinction between data structures:

1. Linear and Non-Linear, 2. Static and Dynamic.

1. Linear vs. Non-Linear Data Structures:

- **Linear:** Elements are arranged sequentially or linearly, where each element is connected to its previous and next element. Examples include Arrays, Linked Lists, Stacks, and Queues. *Real-world example:* A queue of people standing in line at a ticketing counter.
- **Non-Linear:** Elements are arranged in a hierarchical or interconnected network where data items can connect to two or more other items. Examples include Trees and Graphs. *Real-world example:* Company organizational chart (Tree) or social media friendship network (Graph).

2. Static vs. Dynamic Data Structures:

- **Static:** Memory is allocated at compile-time, and its size remains fixed throughout program execution. *Real-world example:* A pre-allocated parking lot with exactly 50 slots painted on the floor.
- **Dynamic:** Memory is allocated at run-time using pointers and heap allocation, allowing the structure to grow or shrink dynamically. *Real-world example:* An expanding seating arrangement at a restaurant where tables can be added or removed as guests arrive.

Q5. Differentiate between static and dynamic data structures with suitable examples.

Parameter	Static Data Structure	Dynamic Data Structure
Memory Allocation	Allocated during compile-time in stack memory.	Allocated during run-time in heap memory.
Size	Fixed and immutable once created.	Flexible; can expand or shrink as needed.
Efficiency	Faster access because memory locations are contiguous and fixed.	Slightly slower access due to pointer traversal and overhead.
Memory Utilization	Risk of memory wastage (if underutilized) or overflow (if full).	Optimal utilization; memory is requested only when needed.
Example	1D / 2D Arrays.	Linked Lists, Binary Search Trees, Graphs.

Q6. Define an algorithm. State its essential characteristics and explain with a simple example.

1. Definition: An algorithm is a finite, unambiguous, step-by-step sequence of well-defined instructions designed to solve a specific computational problem or perform a computation.

2. Essential Characteristics (Knuth's Criteria):

- **Input:** Zero or more quantities are externally supplied.
- **Output:** At least one quantity is produced.
- **Definiteness:** Each instruction must be clear and unambiguous.
- **Finiteness:** The algorithm must terminate after a finite number of steps.
- **Effectiveness:** Every instruction must be basic enough to be carried out, in principle, by a person using only pencil and paper.

3. Example (Algorithm to check if a number is even): Step 1: Read integer n . Step 2: Compute remainder $r = n \bmod 2$. Step 3: If $r == 0$, print "Even". Else, print "Odd". Step 4: Stop.

Q7. What is algorithm complexity analysis? Explain time complexity and space complexity with suitable examples.

1. Algorithm Complexity Analysis: It is the theoretical evaluation of the resources (CPU time and memory space) required by an algorithm to run as a function of the input size n , independent of hardware and compiler fluctuations.

2. Time Complexity: Measures the amount of computer time an algorithm takes to complete as a function of input size n . Counted via fundamental operations executed. *Example:* Linear search takes $O(n)$ time in the worst case.

3. Space Complexity: Measures the total amount of computer memory space required by the algorithm during its execution, including input storage, variables, and auxiliary working space. *Example:* An iterative factorial algorithm has space complexity $O(1)$, whereas a recursive factorial function has $O(n)$ due to call stack frame accumulation.

Q8. Explain the Step Count Method of analyzing time complexity with a suitable example.

1. Concept: The Step Count Method involves building a frequency table that counts the exact number of times each statement in an algorithm is executed relative to the input size n .

2. Example: Matrix Addition or Array Summation

```
Algorithm ArraySum(A, n)
1. Sum = 0;           // executed 1 time
2. For i = 1 to n do   // executed n + 1 times
3.     Sum = Sum + A[i]; // executed n times
4. End For
5. Return Sum;         // executed 1 time
```

Total Steps $T(n)$: $1 + (n + 1) + n + 1 = 2n + 3$. As $n \rightarrow \infty$, lower-order terms and coefficients are dropped, yielding a time complexity of $O(n)$.

Q9. Explain the asymptotic notations used in algorithm analysis: Big-O (O), Theta (Θ), and Omega (Ω) with suitable examples.

- **Big-O Notation (O) - Upper Bound:** Represents the asymptotic upper bound. $f(n) = O(g(n))$ if there exist positive constants c and n_0 such that $0 \leq f(n) \leq c \cdot g(n)$ for all $n \geq n_0$. Example: Bubble sort worst-case is $O(n^2)$.
- **Omega Notation (Ω) - Lower Bound:** Represents the asymptotic lower bound. $f(n) = \Omega(g(n))$ if there exist positive constants c and n_0 such that $0 \leq c \cdot g(n) \leq f(n)$ for all $n \geq n_0$. Example: Comparison-based sorting lower bound is $\Omega(n \log n)$.
- **Theta Notation (Θ) - Tight Bound:** Represents the tight bound (both upper and lower bound). $f(n) = \Theta(g(n))$ if $f(n) = O(g(n))$ and $f(n) = \Omega(g(n))$. Example: Binary search has a tight bound of $\Theta(\log n)$.

Q10. Explain best case, worst case, and average case time complexity with a suitable example.

- **Best Case Complexity:** The minimum time required for an algorithm to execute over all possible inputs of size n (optimal scenario). For Linear Search, if the target element is at the first position $A[0]$, time complexity is $O(1)$.
- **Worst Case Complexity:** The maximum time required for an algorithm to execute over any input of size n (pessimistic scenario). For Linear Search, if the target element is absent or at the last position, time complexity is $O(n)$.
- **Average Case Complexity:** The expected time required over all possible inputs of size n , assuming a uniform probability distribution. For Linear Search, assuming equal probability of finding the element anywhere, average comparisons equal $\frac{n+1}{2}$, yielding $O(n)$.

Q11. Explain the concept of sequential organization in data storage with a suitable example.

1. **Concept:** In sequential organization, data elements are stored in contiguous memory locations in a linear sequence. Accessing the i -th element requires knowing the base address and the fixed size of each element, allowing direct random access via simple pointer arithmetic.
2. **Example:** An array of integers stored in RAM. If integer size is 4 bytes and base address is 2000, the fourth element $A[3]$ is located at $2000 + 3 \times 4 = 2012$.

Q12. Describe how arrays serve as an Abstract Data Type (ADT) with suitable examples.

An array ADT defines a collection of homogeneous elements stored at contiguous memory locations, accessible via integer indices. Operations include:

- `get(index)`: Retrieve element at specified index.
- `set(index, value)`: Update value at index.
- `length()`: Return total capacity.

It provides **$O(1)$** random access time, making it an extremely efficient ADT for indexed collections.

Q13. A 2D array $A[4][5]$ is stored in row-major order with base address 1000 and each element occupying 4 bytes. Calculate the address of element $A[2][3]$. Also, explain how the address would differ under column-major order.

Given: Array $A[4][5]$ (4 rows, 5 columns: $R = 4$, $C = 5$), Base Address $B = 1000$, Element size $W = 4$ bytes, Target element $A[2][3]$ ($i = 2$, $j = 3$, using 0-based indexing).

1. Row-Major Order Formula:

$$\text{ext}\{\text{Address}\}(A[i][j]) = B + W \text{ times } (i \text{ times } C + j)$$

$$\text{ext}\{\text{Address}\}(A[2][3]) = 1000 + 4 \text{ times } (2 \text{ times } 5 + 3) = 1000 + 4 \text{ times } (10 + 3) = 1000 + 4 \text{ times } 13 = 1000 + 52 = 1052.$$

2. Column-Major Order Formula & Calculation:

$$\text{ext}\{\text{Address}\}(A[i][j]) = B + W \text{ times } (j \text{ times } R + i)$$

$$\text{ext}\{\text{Address}\}(A[2][3]) = 1000 + 4 \text{ times } (3 \text{ times } 4 + 2) = 1000 + 4 \text{ times } (12 + 2) = 1000 + 4 \text{ times } 14 = 1000 + 56 = 1056.$$

Thus, row-major order yields address **1052**, whereas column-major order yields **1056**.

Q14. Define an Ordered List. Explain its properties, advantages, and basic operations with examples.

1. Definition & Properties: An Ordered List is a linear list where elements are arranged in a specific sorted order (ascending or descending).

2. Advantages: Enables binary search (**$O(\log n)$** search time) and rapid range queries.

3. Basic Operations: Insertion (maintains sort order), Deletion, Traversal, and Search.

Q15. Explain how a polynomial can be represented using an array. Illustrate with an example.

1. Concept: A polynomial $P(x) = a_n x^n + a_{n-1} x^{n-1} + \dots + a_0$ can be represented using an array where the index of the array corresponds to the exponent of the term, and the value stored at that index corresponds to the coefficient.

2. Example: For $P(x) = 5x^3 + 2x + 7$, an array of size 4 can store coefficients at indices corresponding to degrees 3, 2, 1, 0 as: $A = [5, 0, 2, 7]$.

Unit-II: Searching and Sorting

Q1. Explain the classification of sorting as internal and external sorting. Also, define the concept of sort stability with examples.

- **Internal Sorting:** Performed when the entire dataset fits entirely into primary memory (RAM) simultaneously. Examples: QuickSort, MergeSort, BubbleSort.
- **External Sorting:** Performed when the dataset is too large to fit into RAM, requiring external storage (HDDs/SSDs) with chunking and merging. Example: External Merge Sort.
- **Sort Stability:** A sorting algorithm is stable if it preserves the relative order of records with equal keys. For example, sorting students by grade where student (Alice, B) appears before (Bob, B); a stable sort guarantees Alice remains before Bob in the sorted output.

Q2. Explain the Linear Search algorithm. Write its pseudo-code, analyze its time complexity, and provide a simple example.

1. Procedure: Sequentially checks each element of the list from beginning to end until a match is found or the list ends.

2. Pseudo-code:

```
LinearSearch(A, n, target)
For i = 0 to n-1 do
    If A[i] == target then
        Return i
    End If
End For
Return -1
```

3. Complexity: Best Case $O(1)$, Worst Case $O(n)$, Space Complexity $O(1)$.

Q3. Apply the Sentinel Search technique to search for a given key element in an array of your choice. Write the algorithm and analyze how it reduces the number of comparisons compared to Linear Search.

1. Concept: Sentinel search places the target key at the end (sentinel position) of the array to eliminate the boundary check condition ($i < n$) inside the loop, halving conditional overhead.

2. Algorithm & Analysis: Reduces loop overhead by eliminating the index-out-of-bounds check on every iteration, leading to faster execution times in large datasets.

Q4. Explain the Binary Search algorithm. With simple example.

1. Principle: Operates on sorted arrays by repeatedly dividing the search interval in half. If the target value is less than the middle element, narrow to the lower half; otherwise, upper half.

2. Example: Searching 23 in [2, 5, 8, 12, 16, 23, 38, 56, 72, 91]. Mid is 16 (). $23 > 16$, search right half. Mid is 56, search left half. Found 23 at index 5.

Complexity: Time $O(\log n)$, Space $O(1)$ (iterative).

Q5. Explain the Bubble Sort algorithm with a suitable example. Also, state whether it is stable.

1. Procedure: Repeatedly steps through the list, compares adjacent elements, and swaps them if they are in the wrong order.

2. Stability: Bubble Sort is a **stable** sorting algorithm because equal elements are never swapped.

Q6. Explain the adaptive nature of Bubble Sort and discuss its stability. Illustrate with an example how Bubble Sort maintains stability and adapts to already sorted data.

1. Adaptability: By incorporating a boolean swap flag, Bubble Sort detects if zero swaps occurred during a pass, terminating early in $O(n)$ time for already sorted arrays.

Q7. Explain the Selection Sort algorithm. Write its pseudo-code or C++ program, and discuss its procedure, time and space complexity, and whether it is stable or not.

1. Procedure: Divides the array into sorted and unsorted regions. Repeatedly selects the minimum element from the unsorted region and swaps it with the leftmost unsorted element.

2. Complexity & Stability: Time complexity is $\Theta(n^2)$ across all cases (Best, Average, Worst). Space complexity is $O(1)$. Standard Selection Sort is **unstable** due to long-range swapping.

Q8. Write a C++ program for Selection Sort. Comment on the time complexity of Selection Sort.

```
#include <iostream>
using namespace std;

void selectionSort(int arr[], int n) {
    for (int i = 0; i < n - 1; i++) {
        int minIdx = i;
        for (int j = i + 1; j < n; j++) {
            if (arr[j] < arr[minIdx]) minIdx = j;
        }
        swap(arr[minIdx], arr[i]);
    }
}
```

Comment: Time complexity is $O(n^2)$ because of nested loops scanning the unsorted array, regardless of initial input order.

Q9. Explain the Insertion Sort algorithm. Write its pseudo-code or C++ program, and discuss its procedure, time and space complexity, and whether it is stable and adaptive.

1. Procedure: Builds the final sorted array one item at a time by repeatedly taking the next element and inserting it into its correct position among the previously sorted elements.

2. Characteristics: Time complexity is $O(n)$ best case (already sorted), $O(n^2)$ worst case. It is both **stable** and **adaptive**.

Q10. Define a Heap. Explain the Heap data structure, its types, properties, and basic operations with examples.

1. Definition: A Heap is a specialized tree-based data structure that satisfies the heap property (Max-Heap: parent $\geq$ children; Min-Heap: parent $\leq$ children) and is almost a complete binary tree.

2. Operations: Insertion, Deletion (extract max/min), and Heapify.

Q11. What is a Min Heap and a Max Heap? Sort following elements using Heap Sort in ascending order: 77, 15, 91, 21, 6, 46.

1. Definitions: Max-Heap has parent nodes greater than children; Min-Heap has parent nodes smaller than children.

2. Heap Sort Execution (Ascending Order using Max-Heap):

- Initial Array: **[77, 15, 91, 21, 6, 46]**
- Step 1 (Build Max-Heap): **[91, 21, 77, 15, 6, 46]**
- Step 2 (Extract root and heapify repeatedly): Results in sorted ascending order: **[6, 15, 21, 46, 77, 91]**.

Q12. What is hashing? Explain the concept and role of hashing in efficient searching with a suitable example.

1. Concept: Hashing is a technique used to map data of arbitrary size to fixed-size values (hash keys) using a hash function. It enables near **$O(1)$** average time complexity for search, insertion, and deletion operations.

Q13. Define hashing. Given a hash table of size 7 and the keys (23, 44, 12, 18, 39, 55), apply the division-modulo hash function to insert each key. Resolve any collisions using linear probing and show the final hash table.

1. Hash Function: $h(k) = k \bmod 7$, Hash Table Size $N = 7$ (indices **0** *dots* **6**).

2. Key Insertions:

- **23** $\bmod 7 = 2 \rightarrow$ Insert at index 2.
- **44** $\bmod 7 = 2 \rightarrow$ Collision at index 2! Linear probing places **44** at index 3.
- **12** $\bmod 7 = 5 \rightarrow$ Insert at index 5.
- **18** $\bmod 7 = 4 \rightarrow$ Insert at index 4.
- **39** $\bmod 7 = 4 \rightarrow$ Collision at index 4! Linear probing checks 5 (occupied), places **39** at index 6.
- **55** $\bmod 7 = 6 \rightarrow$ Collision at index 6! Linear probing wraps around to index 0, places **55** at index 0.

3. Final Hash Table:

Index	0	1	2	3	4	5	6
Key	55	Empty	23	44	18	12	39

Q14. Explain different types of hash functions with suitable examples.

- **Division Method:** $h(k) = k \bmod m$ where m is prime.
- **Multiplication Method:** $h(k) = \lfloor m \cdot (k \cdot A \bmod 1) \rfloor$ where $0 < A < 1$.
- **Mid-Square Method:** Square the key, extract middle digits.
- **Folding Method:** Partition key into parts and add them together.

Q15. What are the different collision resolution techniques in hashing? Explain open hashing or separate chaining with an example.

1. **Collision Resolution Techniques:** Broadly classified into Closed Hashing (Open Addressing: Linear Probing, Quadratic Probing, Double Hashing) and Open Hashing (Separate Chaining).
2. **Separate Chaining:** Each slot of the hash table contains a pointer to a linked list of keys that hashed to the same index. When a collision occurs, the new key is simply appended to the linked list at that index.